RQ1.R

Ester Maiocchi (10800862), Marianna Mazza (10838595), Arianna Perotti (10811649)

```r
df <- read.csv("df_clean.csv")
df <- df[,-1]

seed = 2002

# Transform categorical variables into factors
df$Device_Name <- as.factor(df$Device_Name)
df$Brand       <- as.factor(df$Brand)
df$Category    <- as.factor(df$Category)
df$Model       <- as.factor(df$Model)

# Transform water resistance into a factor with ordered levels
unique(df$Water_Resistance_Rating)
```

```
## [1] "3ATM"  "IP68"  "IPX8"  "5ATM"  "IPX4"  "IPX7"  "10ATM"
```

```r
df$Water_Resistance_Rating <- factor(df$Water_Resistance_Rating,
                                     levels = c("IPX4", "3ATM", "IPX7", "5ATM",
"IPX8", "IP68", "10ATM"),
                                     ordered = TRUE)
df$Water_Resistance_Score <- as.numeric(df$Water_Resistance_Rating)

# Transform connectivity into a factor with ordered levels
unique(df$Connectivity_Features)
```

```
## [1] "Bluetooth, WiFi"           "WiFi, Bluetooth, NFC"
## [3] "Bluetooth"                 "WiFi, Bluetooth, NFC, LTE"
```

```r
df$Connectivity_Features <- factor(
  df$Connectivity_Features,
  levels = c("Bluetooth", "Bluetooth, WiFi", "WiFi, Bluetooth, NFC", "WiFi,
Bluetooth, NFC, LTE"),
  ordered = TRUE)
df$Connectivity_Features_Score <- as.numeric(df$Connectivity_Features)

# Transform app ecosystem into a factor with ordered levels
unique(df$App_Ecosystem_Support)
```

```
## [1] "Cross-platform" "iOS"            "Android/iOS"
```

```r
df$App_Ecosystem_Support <- factor(
  df$App_Ecosystem_Support,
  levels = c("iOS", "Android/iOS", "Cross-platform"),
  ordered = TRUE)
df$App_Ecosystem_Support_Score <- as.numeric(df$App_Ecosystem_Support)
```

```r
# RQ1: Which device category offers the best balance of accuracy, battery life and other features for typical users? ----

library(dplyr)
```

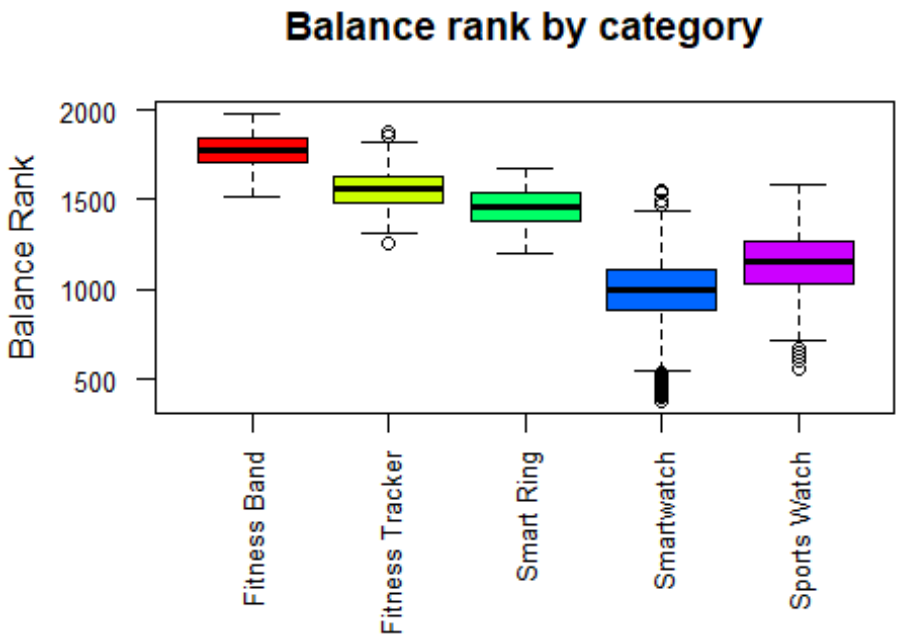
```
##
## Caricamento pacchetto: 'dplyr'
```

```
## I seguenti oggetti sono mascherati da 'package:stats':
##
##     filter, lag
```

```
## I seguenti oggetti sono mascherati da 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
df <- df %>%
  mutate(
    Rank_HeartAcc = rank(-Heart_Rate_Accuracy_Percent),
    Rank_StepAcc  = rank(-Step_Count_Accuracy_Percent),
    Rank_SleepAcc = rank(-Sleep_Tracking_Accuracy_Percent),

    Rank_Battery  = rank(-Battery_Life_Hours),
    Rank_Water    = rank(-Water_Resistance_Score),

    Rank_GPS      = rank(GPS_Accuracy_Meters),
    Rank_Conn     = rank(-Connectivity_Features_Score),
    Rank_Ecosys   = rank(-App_Ecosystem_Support_Score),
    Rank_Count    = rank(-Health_Sensors_Count)
  ) %>%
  mutate(
    Balance_Rank = (
      Rank_HeartAcc + Rank_StepAcc + Rank_SleepAcc +
        Rank_Battery + Rank_Water +
        Rank_GPS + Rank_Conn + Rank_Ecosys + Rank_Count
    ) / 9
  )


df <- df %>% arrange(Balance_Rank)

category_balance <- df %>%
  group_by(Category) %>%
  summarise(
    median_balance = median(Balance_Rank),
    mean_balance   = mean(Balance_Rank),
    sd_balance     = sd(Balance_Rank),
    n = n()
  ) %>%
  arrange(median_balance)
```

```r
# Permutation-based one-way anova ----
B = 5e3
g <- nlevels(df$Category)
n <- dim(df)[1]

par(mar = c(8, 4, 4, 2))
plot(
  df$Category, df$Balance_Rank,
  xlab = " ",
  ylab = "Balance Rank",
  col  = rainbow(g),
  main = "Balance rank by category",
  las  = 2,
  cex.axis = 0.8
)
```

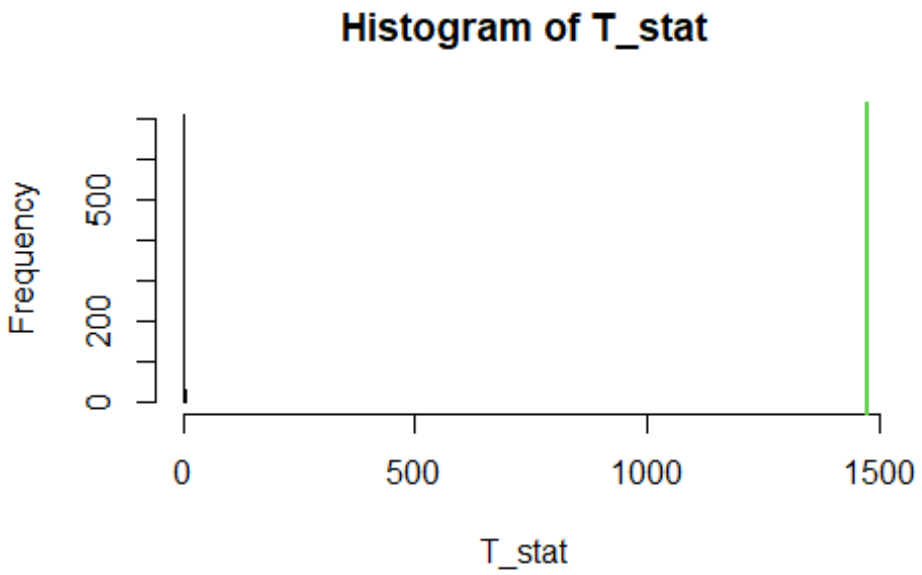



```r
fit <- aov(Balance_Rank ~ Category, data = df)
summary(fit)
```

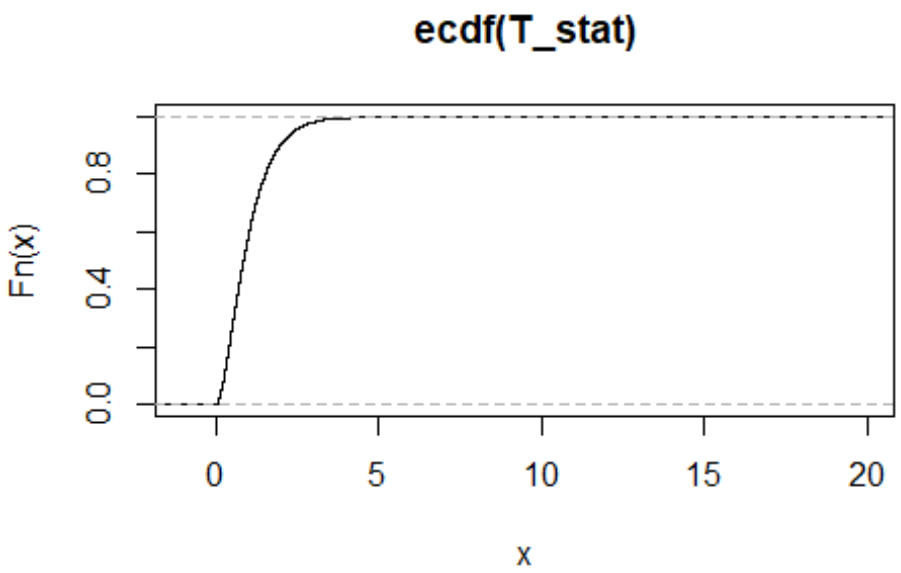
```
##               Df    Sum Sq  Mean Sq F value Pr(>F)
## Category       4 165196574 41299144    1473 <2e-16 ***
## Residuals   2370  66428295    28029
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
T0 <- summary(fit)[[1]][1,4]  # extract the test statistic
T0
```

```
## [1] 1473.453

T_stat <- numeric(B)
n <- dim(df)[1]

for(perm in 1:B){
  # Permutation:
  permutation <- sample(1:n)
  rank_perm <- df$Balance_Rank[permutation]
  fit_perm <- aov(rank_perm ~ df$Category)

  # Test statistic:
  T_stat[perm] <- summary(fit_perm)[[1]][1,4]
}

hist(T_stat,xlim=range(c(T_stat,T0)),breaks=30)
abline(v=T0,col=3,lwd=2)
```




```
plot(ecdf(T_stat),xlim=c(-1,20))
abline(v=T0,col=3,lwd=4)
```




```
p_val <- sum(T_stat>=T0)/B
p_val

## [1] 0

# Permutation test on medians between two categories
perm_median_test <- function(cat1, cat2, data, iter = 10000) {
  # extract the values for the two categories
  x <- data$Balance_Rank[data$Category == cat1]
  y <- data$Balance_Rank[data$Category == cat2]

  # observed statistic
  T0 <- abs(median(x) - median(y))

  T_stat <- numeric(iter)

  # pooled sample
  x_pooled <- c(x, y)
  n <- length(x_pooled)
  n1 <- length(x)

  # permutation loop
  for (perm in 1:iter) {
    perm_idx <- sample(1:n)
    x_perm <- x_pooled[perm_idx]
    x1_perm <- x_perm[1:n1]
    x2_perm <- x_perm[(n1 + 1):n]
    T_stat[perm] <- abs(median(x1_perm) - median(x2_perm)) # permuted statistic
```

```r
  }

  # p-value
  p_val <- mean(T_stat >= T0)
  return(p_val)
}


# Pairwise permutation median tests for all categories
pairwise_perm_median <- function(data, iter = 10000) {
  cats <- unique(data$Category)
  results <- data.frame()
  for (i in 1:(length(cats) - 1)) {
    for (j in (i + 1):length(cats)) {
      cat1 <- cats[i]
      cat2 <- cats[j]
      p_val <- perm_median_test(cat1, cat2, data = data, iter = iter)
      results <- rbind(results,
                       data.frame(
                         Category1 = cat1,
                         Category2 = cat2,
                         p_value = p_val
                       ))
    }
  }
  return(results)
}

results_pairwise <- pairwise_perm_median(df, iter = 10000)
results_pairwise
```

```
##          Category1       Category2 p_value
## 1       Smartwatch    Sports Watch       0
## 2       Smartwatch      Smart Ring       0
## 3       Smartwatch Fitness Tracker       0
## 4       Smartwatch    Fitness Band       0
## 5     Sports Watch      Smart Ring       0
## 6     Sports Watch Fitness Tracker       0
## 7     Sports Watch    Fitness Band       0
## 8       Smart Ring Fitness Tracker       0
## 9       Smart Ring    Fitness Band       0
## 10 Fitness Tracker    Fitness Band       0
```

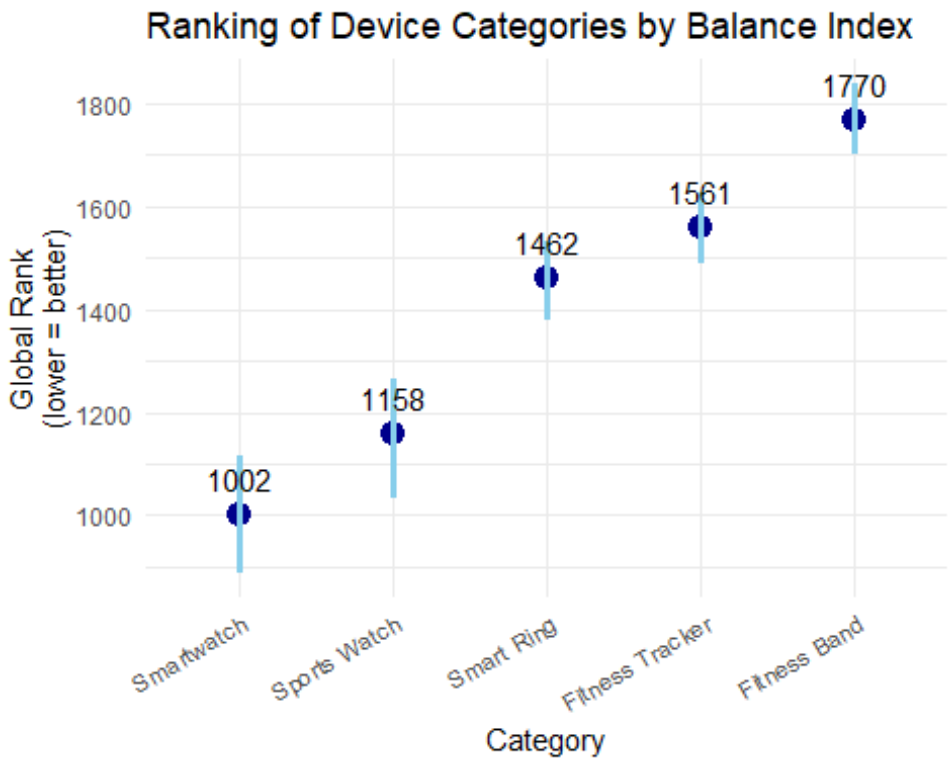
```r
library(reshape2)
library(ggplot2)


# Summary plot data for categories
category_summary <- df %>%
  group_by(Category) %>%
  summarise(
```

```
    median_balance = median(Balance_Rank),
    mean_balance = mean(Balance_Rank),
    q1 = quantile(Balance_Rank, 0.25),
    q3 = quantile(Balance_Rank, 0.75)
  ) %>%
  arrange(median_balance)

category_summary
```

```
## # A tibble: 5 × 5
##   Category        median_balance mean_balance    q1    q3
##   <fct>                    <dbl>        <dbl> <dbl> <dbl>
## 1 Smartwatch                1002         995.  888. 1116.
## 2 Sports Watch              1158.       1144. 1033  1264.
## 3 Smart Ring                1462.       1458. 1378. 1534.
## 4 Fitness Tracker           1561.       1562. 1489. 1627.
## 5 Fitness Band              1770.       1768. 1703. 1838.
```

```
ggplot(category_summary,
       aes(x = reorder(Category, median_balance),
           y = median_balance)) +
  geom_point(size = 4, color = "darkblue") +
  # Interquartile range line
  geom_linerange(aes(ymin = q1, ymax = q3), size = 1.2, color = "skyblue") +
  geom_text(aes(label = sprintf("%.0f", median_balance)),
            vjust = -1, size = 4) +
  labs(title = "Ranking of Device Categories by Balance Index",
       x = "Category",
       y = "Global Rank\n(lower = better)") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# Smartwatch vs Sports Watch ----

# Select the continuous metrics of interest
metrics <- c(
  "Heart_Rate_Accuracy_Percent",
  "Step_Count_Accuracy_Percent",
  "Sleep_Tracking_Accuracy_Percent",
  "Battery_Life_Hours",
  "GPS_Accuracy_Meters",
  "Health_Sensors_Count"
)

# Filter the dataframe to include only Smartwatch and Sports Watch
df_two <- df %>% filter(Category %in% c("Smartwatch", "Sports Watch"))


# Perform Wilcoxon rank-sum test (Mann-Whitney) for all selected metrics
wmw_results <- data.frame(
  Metric = metrics,
  p_value = sapply(metrics, function(m) {
    wilcox.test(
      df_two[[m]][df_two$Category == "Smartwatch"],
      df_two[[m]][df_two$Category == "Sports Watch"],
      exact = FALSE
    )$p.value
  })
```

```
)
wmw_results
```

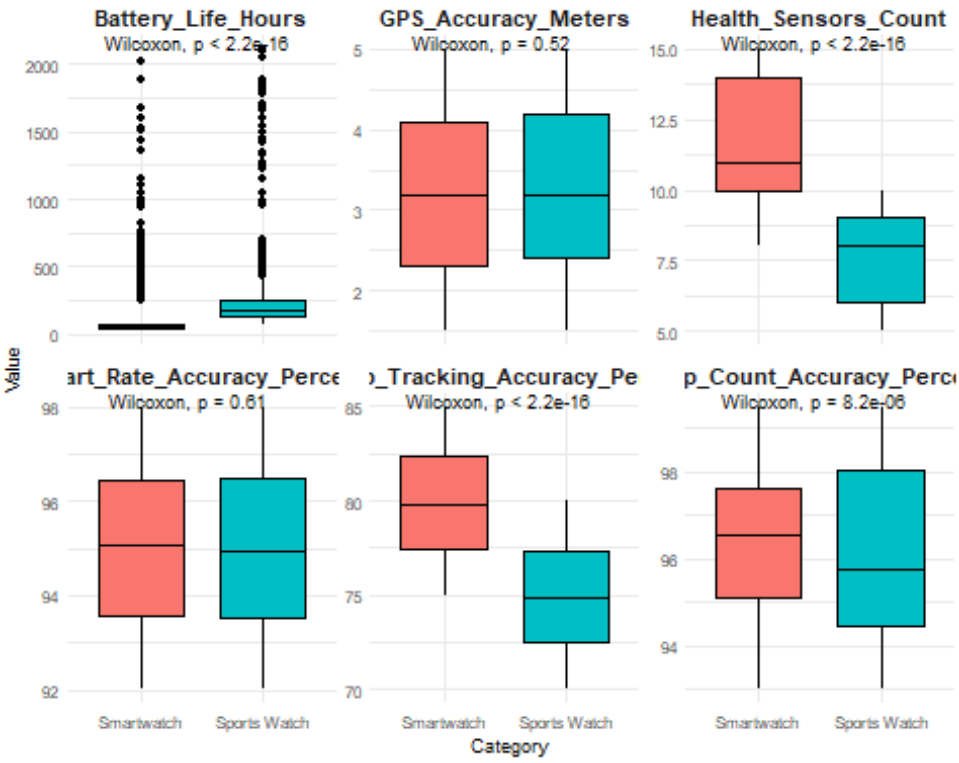
```
##                                                                 Metric       p_value
## Heart_Rate_Accuracy_Percent         Heart_Rate_Accuracy_Percent  6.126405e-01
## Step_Count_Accuracy_Percent         Step_Count_Accuracy_Percent  8.159239e-06
## Sleep_Tracking_Accuracy_Percent Sleep_Tracking_Accuracy_Percent 5.557504e-134
## Battery_Life_Hours                               Battery_Life_Hours 3.361673e-159
## GPS_Accuracy_Meters                             GPS_Accuracy_Meters  5.161311e-01
## Health_Sensors_Count                           Health_Sensors_Count 5.682134e-152
```

```
library(ggpubr)
library(ggplot2)
library(tidyr)
```

```
##
## Caricamento pacchetto: 'tidyr'
```

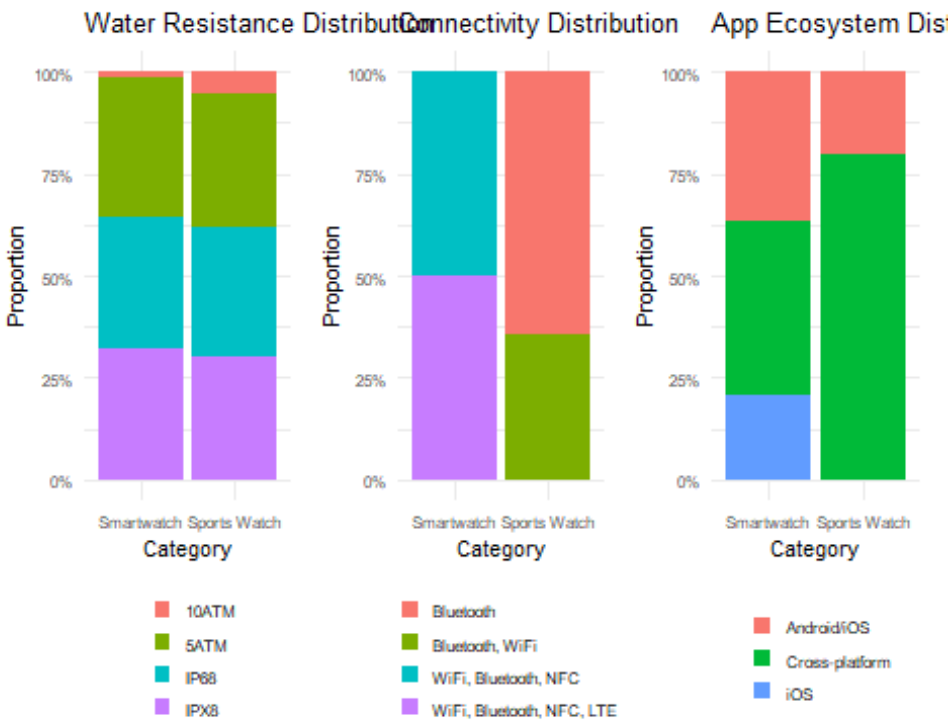
```
## Il seguente oggetto è mascherato da 'package:reshape2':
##
##     smiths
```

```
df_two_long <- df_two %>%
  pivot_longer(cols = all_of(metrics), names_to = "Metric", values_to = "Value")

ggboxplot(df_two_long, x = "Category", y = "Value", fill = "Category") +
  stat_compare_means(method = "wilcox.test") +
  facet_wrap(~ Metric, scales = "free_y") +
  theme_minimal(base_size = 7) +
  theme(
    strip.text = element_text(size = 8, face = "bold"),
    axis.title = element_text(size = 7),
    axis.text  = element_text(size = 6),
    legend.position = "none"
  )
```




```r
# Categorical metrics of interest
library(patchwork)
df_wr <- df_two %>%
  mutate(Water_Resistance_Rating = as.character(Water_Resistance_Rating))
df_con <- df_two %>%
  mutate(Connectivity_Features = as.character(Connectivity_Features))
df_eco <- df_two %>%
  mutate(App_Ecosystem_Support = as.character(App_Ecosystem_Support))

p1 <- ggplot(df_wr, aes(x = Category, fill = Water_Resistance_Rating)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  guides(fill = guide_legend(ncol = 1)) +
  labs(
    title = "Water Resistance Distribution",
    y = "Proportion",
    x = "Category",
    fill = ""
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(size = 10),
    axis.title = element_text(size = 8),
    axis.text = element_text(size = 6),
    legend.text = element_text(size = 6),
    legend.key.size = unit(0.25, "cm")
  )
```

```r
p2 <- ggplot(df_con, aes(x = Category, fill = Connectivity_Features)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  guides(fill = guide_legend(ncol = 1)) +
  labs(
    title = "Connectivity Distribution",
    y = "Proportion",
    x = "Category",
    fill = ""
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(size = 10),
    axis.title = element_text(size = 8),
    axis.text = element_text(size = 6),
    legend.text = element_text(size = 6),
    legend.key.size = unit(0.25, "cm")
  )

p3 <- ggplot(df_eco, aes(x = Category, fill = App_Ecosystem_Support)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  guides(fill = guide_legend(ncol = 1)) +
  labs(
    title = "App Ecosystem Distribution",
    y = "Proportion",
    x = "Category",
    fill = ""
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title = element_text(size = 10),
    axis.title = element_text(size = 8),
    axis.text = element_text(size = 6),
    legend.text = element_text(size = 6),
    legend.key.size = unit(0.25, "cm")
  )

(p1 + p2 + p3) & theme(legend.position = "bottom")
```




```r
# Permutation-based chi-square tests for water resistance, connectivity and ecosystem
df2 <- df %>%
  dplyr::filter(Category %in% c("Smartwatch", "Sports Watch")) %>%
  droplevels()


perm_chisq_test <- function(group, x, B = 10000) {

  if (!is.null(seed)) set.seed(seed)

  tab0 <- table(group, x)

  T0 <- chisq.test(tab0, correct = FALSE)$statistic

  T_perm <- numeric(B)
  n <- length(group)

  for (b in 1:B) {
    group_perm <- sample(group)
    tab_perm <- table(group_perm, x)

    T_perm[b] <- chisq.test(tab_perm, correct = FALSE)$statistic
  }

  p_val <- mean(T_perm >= T0)
```

```r
  return(list(
    T0 = as.numeric(T0),
    T_perm = T_perm,
    p_value = p_val
  ))
}


res_water <- perm_chisq_test(
  group = df2$Category,
  x = df2$Water_Resistance_Rating,
  B = 10000
)

res_water$p_value
```

```
## [1] 1e-04
```

```r
res_connectivity <- perm_chisq_test(
  group = df2$Category,
  x = df2$Connectivity_Features,
  B = 10000
)

res_connectivity$p_value
```

```
## [1] 0
```

```r
res_ecosystem <- perm_chisq_test(
  group = df2$Category,
  x = df2$App_Ecosystem_Support,
  B = 10000
)

res_ecosystem$p_value
```

```
## [1] 0
```